POLYTECH NICE SOPHIA
Département Robotique et Systèmes Autonomes

RAPPORT DE STAGE ASSISTANT INGÉNIEUR

Stage de quatrième année du cycle ingénieur

Mise en oeuvre d'un robot AgileX

Maxime DELUBAC-LOPPI--DUMORTIER

Nom de l'entreprise d'accueil : Polytech'Lab
Adresse de l'entreprise : Campus SophiaTech, 930 Route des Colles, 06903 SOPHIA ANTIPOLIS
Responsable de stage (Entreprise) : Pascal MASSON
Adresse de l'école : Campus SophiaTech, 930 Route des Colles, 06903 SOPHIA ANTIPOLIS
Tuteur (École) : Delphine CALLAGHAN, Directrice Adjointe vie étudiante et communication, référente racisme, antisémitisme et discriminations, professeure d'anglais
Période de stage : du 20 avril au 7 août 2026

Année universitaire 2025-2026

Remerciements

Je tiens tout d'abord à remercier Monsieur Pascal Masson, enseignant-chercheur et responsable de mon stage, pour m'avoir accueilli au sein de Polytech'Lab. Ses conseils avisés, sa disponibilité et la confiance qu'il m'a accordée tout au long de ces mois ont été d'une aide précieuse pour mener à bien la mise en œuvre du robot AgileX.

J'aimerai aussi fortement remercier Frédéric Juan qui s'occupe du Fablab de Polytech, pour m'avoir aidé à trouver cette opportunité de stage.

Mes remerciements s'adressent également à mon camarade de classe et collègue de stage Thomas Chavaroche, ainsi qu'à mon manager Matthias Vermot-Desroches, avec qui j'ai pu travailler dans une excellente ambiance.

Finalement, je remercie Elena.

Sommaire

Remerciements **1**
Sommaire **2**
Introduction **4**
1.1 Contexte 4
1.2 Objectifs du projet 4
1.3 Annonce du plan 5
Prise en main, Capteurs et Contraintes Matérielles **5**
2.1 Environnement de travail 5
2.1.1 Architecture logicielle et infrastructure système 5
2.1.2 Configuration réseau et connectivité Teltonika 6
2.1.3 Paquets ROS 2 pré-installés et Workspace 6
2.1.4 Difficultés de mise en œuvre 6
2.2 Matériel et architecture du robot 7
2.2.1 Spécifications matérielles 7
2.2.2 Principe de fonctionnement de la caméra Intel Real-Sense 7
2.2.2.1 Spécifications matérielles 7
2.2.2.2 Conventions géométriques 7
2.2.2.3 Analyse détaillée des topics ROS 2 publiés 8
2.2.3 Support caméra imprimé en 3D 8
2.3 Le défi du CPU 9
2.3.1 Constat initial et problématique de performance 9
2.3.2 Diagnostic et identification du problème d'isolation Docker 9
2.3.3 Analyse approfondie de la cause racine : Le conflit de GID (render) 9
2.3.4 Procédure complète de débridage et de configuration 10
2.3.5 Optimisation du stockage et persistance des dépendances Python/APT 10
2.3.6 Intégration logicielle et résultats 11
Localisation par GPS RTK **11**
3.1 Architecture de fusion de capteurs 11
3.2 Correction RTK et client NTRIP 11
3.3 Résultats et limites 12
Vision 2D et 3D **12**
4.1 Environnement ROS2 et outil de détection YOLO 12
4.1.1 Principes fondamentaux du modèle YOLO (You Only Look Once) 12
4.1.2 Mise en place d'un package ROS2 12
4.2 Détection 2D 13
4.2.1 Fonctionnement du noeud ROS2 2D 13
4.3 Détection 3D 13
4.3.1 Problématique du passage de la 2D à la 3D et synchronisation multi-capteurs 13
4.3.2 Mathématiques, Projections et Optimisations Algorithmiques 14
4.3.2.1 Modélisation géométrique : Le modèle du sténopé 14
4.3.2.2 La rétro-projection 2D 3D 14
4.3.2.3 Extraction robuste de la profondeur dans la ROI et Filtrage Médian 15
4.3.2.4 Calcul des dimensions réelles 3D 16

4.3.3 Architecture des topics ROS 2 publiés..... 16
Intelligence artificielle..... 17
5.1 Data Engineering et Datasets..... 17
5.1.1 Dataset BDK100..... 17
5.1.2 Création d'un dataset personnalisé (Roboflow)..... 17
5.1.2.1 Utilisation de Roboflow et nécessité d'un dataset sur-mesure..... 17
5.1.2.2 Évolution de la classification..... 18
5.2 Entraînement de YOLOv11..... 18
5.2.1 Techniques de régularisation et d'augmentation de données..... 19
5.2.2 Optimisation et Déploiement (OpenVINO)..... 19
5.2.3 analyse de la couleur des feux tricolores..... 19
Tests, Validation et Résultats..... 20
6.1 Construction de la piste de test..... 20
6.2 Validation de la détection en conditions réelles..... 20
6.3 Résultats du dataset Roboflow..... 21
6.4 Tests finaux et présentation..... 22
Ingénierie Pédagogique et Transmission..... 22
7.1 L'objectif pédagogique..... 22
7.2 Création des supports..... 23
7.3 Développement de l'environnement étudiant..... 23
7.3.1 Squelettes de code à trous..... 23
7.3.2 Chaîne d'outils et scripts d'automatisation..... 23
7.4 Bêta-testing et validation..... 23
Conclusion..... 25
Annexe..... 25
Glossaire..... 26
Résumé/Abstract..... 30

Les termes techniques marqués d'un astérisque (*) lors de leur première occurrence dans ce rapport sont définis dans le glossaire, en fin de document.

Introduction

1.1 Contexte

Mon stage d'assistant ingénieur de quatrième année, réalisé au sein du département Robotique Autonome de Polytech Nice Sophia, s'est déroulé au FabLab*, un laboratoire en collaboration avec l'école, dédié à l'innovation, qui se situe au milieu du campus. L'école venait d'acquérir des robots mobiles AgileX Scout Mini. Pour aider les étudiants et les chercheurs à les utiliser facilement, l'objectif de notre mission était de rédiger une documentation technique complète et de développer une application robotique autonome afin de combler le manque de ressources existantes. Puisque je fais partie du département robotique en tant qu'élève, j'ai déjà un peu d'expérience dans ce domaine.



Figure 1 : Photo du robot AgileX Scout Mini

1.2 Objectifs du projet

Le projet confié s'articulait autour de deux grands axes principaux :

- L'axe pédagogique : Concevoir une documentation technique claire et des supports de Travaux Pratiques (TP). L'objectif était de permettre aux futurs étudiants et aux équipes du département de prendre en main le robot AgileX Scout Mini plus facilement et de manière autonome.

- L'axe technique et applicatif : Mettre en place et fiabiliser l'environnement de développement, puis développer des fonctionnalités concrètes de navigation et de suivi de ligne pour valider les capacités du robot. En effet l'objectif final, en parallèle de celui de la documentation, était de permettre au robot d'être complètement autonome sur une route en intérieur que nous avons mis en place (lignes à suivre, panneaux de signalisation miniature et feux).

1.3 Annonce du plan

Ce rapport s'articule en sept parties. La partie 2 présente la prise en main du robot, son architecture matérielle et logicielle, ainsi que les difficultés de configuration rencontrées lors de la mise en place de l'environnement. La partie 3 détaille les travaux menés autour de la localisation par GPS RTK*. La partie 4 aborde la détection d'objets en 2D puis en 3D à partir de la caméra de profondeur, avec la modélisation mathématique associée. La partie 5 traite de l'intelligence artificielle : constitution des jeux de données, entraînement et déploiement du modèle YOLOv11*. La partie 6 présente les tests et résultats obtenus sur la piste construite pour l'occasion. La partie 7, enfin, décrit la démarche pédagogique et les travaux pratiques conçus à destination des futurs étudiants.

Prise en main, Capteurs et Contraintes Matérielles

2.1 Environnement de travail

2.1.1 Architecture logicielle et infrastructure système

La prise en main du robot mobile nécessite la mise en place d'un environnement de développement distribué et conteneurisé afin d'assurer la portabilité, la répétabilité des traitements et l'isolation des dépendances logicielles.

L'architecture globale s'appuie sur deux niveaux d'exécution principaux :

- Le système hôte : Une distribution Ubuntu* 24.04 LTS (Noble Numbat) s'exécutant directement sur le CPU* embarqué du robot.
- L'environnement conteneurisé : Un conteneur* Docker* reposant sur une image* Ubuntu 22.04 LTS (Jammy Jellyfish) intégrant le middleware* ROS 2 Humble Hawksbill.

L'inspection de la configuration matérielle et logicielle réalisée via l'outil neofetch qui confirme l'exécution d'un noyau Linux 6.x sous Ubuntu 22.04 à l'intérieur du conteneur.

L'environnement de développement local est articulé autour de VSCode sur la station de travail distante (mon ordinateur), connecté au robot via une extension SSH* Remote. Pour les besoins de visualisation graphique (affichage des flux vidéo, nuages

de points et interfaces RViz2), un serveur d'affichage distant NoVNC* est configuré, permettant d'accéder au bureau virtuel du conteneur via n'importe quel navigateur web.

2.1.2 Configuration réseau et connectivité Teltonika

L'accès sans fil au robot s'effectue au travers d'un routeur industriel 4G/Wi-Fi Teltonika*. La connexion au réseau local présente une contrainte opérationnelle spécifique : l'adresse MAC du point d'accès Wi-Fi (BSSID*) change à chaque réinitialisation ou changement de cellule réseau. Pour établir et stabiliser la connexion réseau avec le robot, la procédure standard suivante doit être exécutée :

- Réaliser un scan complet des réseaux Wi-Fi environnants depuis l'interface d'administration du Teltonika.
- Copier le nouveau BSSID identifié lors du scan.
- Éditer le profil de connexion réseau actif, coller le nouveau BSSID dans le champ dédié.
- Appliquer les modifications

En cas d'instabilité DNS* ou de blocages réseau lors des reconstructions Docker sur le réseau universitaire, la directive --network=host doit être ajoutée aux commandes de build et d'exécution du conteneur (build_and_run.sh) pour partager directement la pile réseau du système hôte.

2.1.3 Paquets ROS 2 pré-installés et Workspace

Le workspace* ROS 2 principal (nommé roskit_ws) qui est implémenté dans le robot, intègre un ensemble de packages* fondamentaux développés et intégrés par les vendeurs. Le robot est prêt à l'utilisation lors de son obtention. On peut donc développer les fonctionnalités qui nous intéressent sans se soucier d'une mise en place complexe.

2.1.4 Difficultés de mise en œuvre

La mise en place initiale de l'environnement s'est heurtée à plusieurs difficultés de configuration. Un premier problème est apparu lors du lancement de l'interface graphique RViz2 : le conteneur ne parvenait pas à afficher les flux vidéo, le serveur d'affichage référencé dans le conteneur ne correspondant pas à celui réellement actif sur l'hôte. La résolution a consisté à vérifier explicitement la variable d'environnement et à la fixer à la valeur correcte dans le fichier de lancement du conteneur, ou alors manuellement si le problème persiste.

Un second problème est survenu lors du lancement du driver de la caméra RealSense : le nœud* ROS 2 échouait silencieusement, la caméra étant déjà accaparée par un processus actif côté hôte. Ce problème à aussi été corrigé dans les fichiers du conteneur. Un troisième problème est apparu après l'installation des pilotes graphiques Intel via le PPA* kobuk-team : celle-ci a entraîné une mise à jour de dépendances système ayant corrompu l'installation de Bash sur l'hôte, provoquant la perte de la coloration syntaxique et de l'autocomplétion dans le terminal. Malgré les recherches de solutions, aucune n'a été retenue. Le lancement du conteneur se faisait en écrivant la commande complète à la main (*docker exec -it bash [nom du conteneur]*).

Enfin, une incompatibilité de version entre NumPy et la bibliothèque cv_bridge* (compilée pour NumPy 1.x) a provoqué des erreurs d'exécution lors du traitement des images, résolue par un downgrade explicite.

2.2 Matériel et architecture du robot

2.2.1 Spécifications matérielles

Le robot est organisé en deux parties :

- L'unité de calcul : un Asus NUC 15 Pro, un ordinateur embarqué qui contient l'environnement de travail sous ROS2*. Il possède un CPU intégré intel récent qui lui permet d'être performant.
- Les capteurs : le robot possède un capteur LIDAR* 3D Robosense Helios 16P, une caméra RGB*-D* Intel Real-Sense D435, un IMU* Phidgetspatial, un GPS RTK simpleRTK2B et un routeur wifi/4G Teltonika.

Il est possible de contrôler le robot manuellement grâce à la télécommande AgileX fournie à l'achat du robot.

2.2.2 Principe de fonctionnement de la caméra Intel Real-Sense

2.2.2.1 Spécifications matérielles

Mes travaux se sont principalement concentrés sur l'utilisation de la caméra de profondeur Intel intégrée au robot. Ce capteur actif combine :

- Un capteur d'image couleur RGB
- Deux capteurs infrarouges (IR) formant un couple stéréoscopique
- Un projecteur de motifs infrarouges non visible destiné à projeter de la texture sur les surfaces homogènes afin de faciliter l'appariement de points et le calcul de disparité*.

La mesure de profondeur s'appuie sur le principe de la stéréovision passive assistée par infrarouge : la profondeur Z d'un point est inversement proportionnelle à la disparité d mesurée entre les deux capteurs IR :

$$Z = \frac{f * B}{d}$$

où f représente la distance focale de la caméra et B la ligne de base (baseline*) séparant les deux projecteurs IR.

2.2.2.2 Conventions géométriques

Les conventions géométriques sont les suivantes :

- L'origine $(0, 0)$ est localisée au coin supérieur gauche du capteur.
- L'axe $+ X$ est orienté vers la droite de l'image.
- L'axe $+ Y$ pointe vers le bas de l'image.

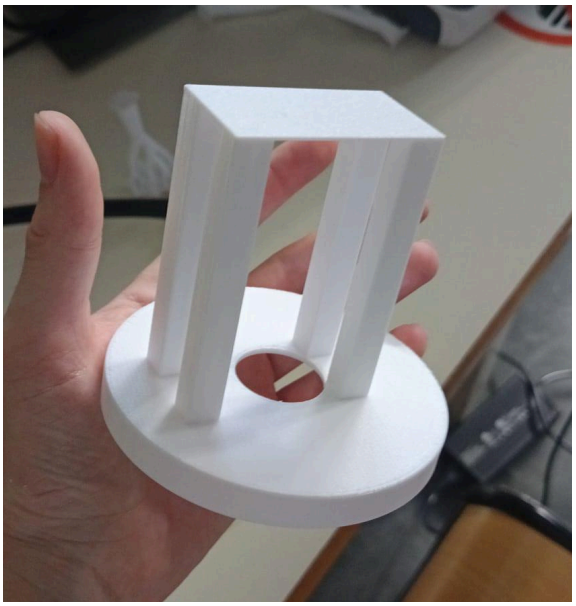
- L'axe $+Z$ est aligné avec l'axe optique, pointant vers l'avant (perpendiculaire au plan de l'image).

2.2.2.3 Analyse détaillée des topics ROS 2 publiés

Le driver realsense2_camera publie un ensemble de topics* couvrant à la fois les flux d'image bruts (couleur et profondeur), les métadonnées de calibration intrinsèque et extrinsèque, ainsi que les transformations géométriques reliant les différents capteurs de la caméra. Les topics d'image (image_raw, image_rect_raw) exposent des matrices dont la structure dépend de l'encodage : une image couleur de résolution 640×480 est ainsi transmise sous la forme d'une matrice aplatie de taille 640×480×3 (trois canaux RGB par pixel), tandis que la carte de profondeur ne contient qu'une seule valeur par pixel. Les topics camera_info fournissent quant à eux la matrice intrinsèque* de calibration, indispensable aux calculs de projection développés en section 3.2.2.

2.2.3 Support caméra imprimé en 3D

En complément de la caméra RealSense D435 intégrée au robot, une seconde caméra a été installée pour Thomas Chavaroche pour ses propres travaux de suivi de ligne, celle-ci offrant un angle de vue plus bas et mieux adapté à cette tâche que la caméra Intel native, positionnée trop haut sur le châssis. Afin de fixer solidement cette caméra, un support a été modélisé sous Fusion 360*, puis imprimé en PETG*, un matériau standard suffisant au regard des contraintes mécaniques de cette application. Le support à été imprimé sur une imprimante 3D Prusa MK4 du FabLab.





Figures 2 et 3 : Photo du support en PETG

2.3 Le défi du CPU

2.3.1 Constat initial et problématique de performance

Lors des premiers essais d'exécution de modèles d'intelligence artificielle pour la détection d'objets (YOLO), le robot ne disposait d'aucune carte graphique dédiée (absence de GPU* NVIDIA). L'exécution des réseaux de neurones s'effectuait intégralement sur le processeur (CPU Intel), ce qui entraînait un ralentissement majeur : la fréquence d'inférence tombait sous la barre des 2 à 3 images par seconde (FPS*), rendant tout contrôle en temps réel impossible.

L'architecture matérielle du processeur Intel intègre néanmoins un processeur graphique intégré (Intel Integrated Graphics / GPU Arc) ainsi qu'un NPU* (Neural Processing Unit) sur les architectures récentes. Cependant, par défaut, le conteneur Docker ROS 2 exécutait l'inférence sur le CPU sans exploiter les capacités d'accélération matérielle de l'iGPU* Intel.

2.3.2 Diagnostic et identification du problème d'isolation Docker

Afin d'exploiter le processeur graphique intégré, le SDK* Intel OpenVINO* a été retenu en association avec la bibliothèque Ultralytics* sous Python* 3. Un script de vérification nommé test_intel.py a été exécuté dans le workspace ROS 2.

Résultat du diagnostic : Le script a retourné uniquement le périphérique 'CPU'. Le périphérique GPU Intel était totalement invisible à l'intérieur du conteneur.

2.3.3 Analyse approfondie de la cause racine : Le conflit de GID (render)

En environnement Linux, la gestion des privilèges d'accès aux cartes graphiques et aux accélérateurs matériels s'appuie sur le système de cartes/groupes. Le fichier périphérique /dev/dri/renderD128 (dédié aux calculs GPU et OpenCL*) appartient à l'utilisateur root et au groupe render. Pour qu'un processus puisse exploiter ce périphérique, l'utilisateur doit appartenir à ce groupe.

Une inspection poussée des identifiants de groupe (GID*) entre l'hôte et le conteneur a révélé une discordance invisible au premier abord :

- Sur le système hôte (Ubuntu 24.04 Noble), la commande stat /dev/dri/renderD128 a montré que le groupe render possédait le GID 992 (attribué dynamiquement lors de l'installation des drivers).
- Dans le conteneur Docker (Ubuntu 22.04 Jammy), le groupe render pré-existant possédait le GID 109 (GID par défaut attribué dans Jammy).

Même si le nom du groupe (render) était identique dans les deux environnements, le noyau Linux effectue les contrôles d'accès sur la valeur numérique du GID. La comparaison des GID entraînait un refus d'accès systématique et silencieux au GPU Intel à l'intérieur du conteneur.

2.3.4 Procédure complète de débridage et de configuration

Pour lever cette contrainte et "débrider" l'accès à l'iGPU, une solution méthodique en quatre étapes a été mise en œuvre :

Étape 1 : Synchronisation du GID dans le Dockerfile*

Afin de faire correspondre exactement les identifiants de groupe, les lignes suivantes ont été insérées dans le Dockerfile avant l'installation des dépendances :

```
# Synchronisation du GID du groupe render avec celui de l'hôte
ARG RENDER_GID=992
RUN groupmod -g ${RENDER_GID} render || groupadd -g ${RENDER_GID}
        render
RUN usermod -a -G render $USER
```

Étape 2 : Transmission dynamique du GID dans build_and_run.sh

Pour éviter de hardcoder le GID (qui peut varier selon la machine hôte), le script de build et de lancement build_and_run.sh a été modifié :

Dans la section docker_build : Extraction automatique du GID de l'hôte via la commande getent :

```
--build-arg RENDER_GID=$(getent group render | cut -d: -f3)
```

Dans la section docker_run : Ajout explicite des permissions d'accès au groupe matériel :

```
--group-add 992 \
--device /dev/dri:/dev/dri \
```

Étape 3 : Installation des pilotes Intel mis à jour sur l'Hôte (PPA Kobuk-Team)

Sur l'hôte Ubuntu 24.04 (Noble), le dépôt Intel officiel présentait un conflit de paquets entre libigc1 et libigc2. La solution a consisté à utiliser le PPA kobuk-team/intel-graphics, maintenu spécifiquement par Canonical pour fournir les derniers pilotes Intel Compute / Level Zero* sans casser les dépendances système.

L'exécution de clinfo | grep "Device Name" a permis de valider que l'hôte détectait correctement les sous-périphériques graphiques Intel.

Étape 4 : Alignement des dépendances GPU dans le conteneur Docker

Dans le Dockerfile, le dépôt officiel Intel pour Ubuntu 22.04 (Jammy) a été ajouté en épinglant la version des paquets (build 1146) pour empêcher toute instabilité.

2.3.5 Optimisation du stockage et persistance des dépendances Python/APT

Trois répertoires de cache persistants ont été créés sur le système hôte afin de survivre aux reconstructions du conteneur Docker : pip-cache, qui conserve les paquets Python (.whl) déjà téléchargés pour éviter leur retéléchargement ; apt*-cache, qui stocke de la même manière les paquets système (.deb) ; et python-packages, qui contient les bibliothèques Python elles-mêmes une fois installées (Ultralytics, OpenVINO, PyTorch, soit plus de 5 Go de dépendances). Ces trois dossiers sont

montés en volumes dans le conteneur via le script build_and_run.sh, garantissant que les paquets lourds ne sont installés qu'une seule fois, quel que soit le nombre de reconstructions ultérieures du conteneur.

2.3.6 Intégration logicielle et résultats

Pour maximiser l'efficacité de l'inférence IA sur le processeur et son iGPU, deux bibliothèques système complémentaires ont été intégrées :

- libtbb12 (Threading Building Blocks) : Bibliothèque développée par Intel permettant de subdiviser les tâches de calcul parallèle et de les répartir efficacement entre les cœurs du CPU et le NPU.
- intel-opencl-icd : Pilote fournissant l'interface standard OpenCL pour le calcul hétérogène sur iGPU.

Le modèle YOLOv11n (préféré à YOLOv8n pour son architecture mieux optimisée pour l'inférence sur processeurs légers) a été exporté au format OpenVINO.

En ré-exécutant test_intel.py à la fin du processus, la sortie confirmait la présence du périphérique GPU.

Grâce à cette chaîne d'optimisation (Drivers Intel OpenCL + Alignement GID + Export OpenVINO + exécution sur iGPU), le temps d'inférence par image a été divisé par plus de 8, permettant d'atteindre un flux d'analyse temps réel fluide (de ~25 à 30 FPS) directement sur le matériel embarqué du robot.

Localisation par GPS RTK

3.1 Architecture de fusion de capteurs

Le robot embarque un module GPS RTK simpleRTK2B, exploitant la technologie GNSS* avec possibilité de correction RTK pour atteindre une précision centimétrique. Afin d'exploiter ce capteur, un package dédié, navigation_pkg, a été développé. Il combine deux composants logiciels :

- un filtre de Kalman étendu , fusionnant les données d'odométrie* des roues et les vitesses angulaires/accélérations de l'IMU, afin d'estimer la position et l'orientation du robot de manière robuste
- un nœud navsat_transform*, chargé de convertir les coordonnées géographiques (latitude/longitude) fournies par le GPS en coordonnées cartésiennes exploitables dans le repère local du robot

Deux fichiers de lancement ROS2 orchestrent cette architecture : localization.launch.py, qui démarre les nœuds de fusion de capteurs, et navigation.launch.py, qui contient les paramètres de contrôle (rayon du robot, vitesse maximale, tolérances d'approche du but, coordonnées de l'objectif)

3.2 Correction RTK et client NTRIP

Le GPS seul offre une précision de localisation de l'ordre de quelques mètres (statut 0 du topic /fix), insuffisante pour une navigation fine sur une piste miniature.

L'obtention d'une précision centimétrique (statut 2, mode RTK) nécessite la réception de corrections différentielles RTCM3* depuis une station de référence proche.

Un client NTRIP* a été développé spécifiquement pour cette tâche, se connectant au réseau de stations de référence Centipède (caster.centipede.fr) et transmettant les corrections reçues au récepteur GPS via un topic /rtcm dédié. La bibliothèque système rtklib a également été installée pour compléter le traitement des corrections RTK côté récepteur.

3.3 Résultats et limites

Malgré la mise en place complète de la chaîne logicielle de localisation (filtre de Kalman, client NTRIP, etc), la stabilisation de la correction RTK et l'obtention d'une précision centimétrique n'ont pas pu être menées à leur terme. L'isolation du bâtiment et les conditions extérieures ont complexifié la recherche sur le GPS. Le calendrier du projet a nécessité de prioriser le développement de la chaîne de détection visuelle 2D/3D, jugée plus critique pour l'objectif final de robot autonome sur route. La finalisation de la localisation GPS RTK a donc été transmise à Matthias Vermot-Desroches, qui a repris ce travail car il avait déjà travaillé sur les filtres de Kalman.

Vision 2D et 3D

4.1 Environnement ROS2 et outil de détection YOLO

4.1.1 Principes fondamentaux du modèle YOLO (You Only Look Once)

L'architecture YOLO (You Only Look Once) repose sur un principe de détection en une seule passe : contrairement aux approches classiques en deux étapes (proposition de régions puis classification), un unique passage du réseau de neurones sur l'image entière suffit à prédire simultanément l'ensemble des boîtes englobantes et les probabilités de classe associées.

Le choix de la variante YOLOv11n (Nano) plutôt que YOLOv8n s'explique par une architecture plus récente, mieux optimisée pour l'inférence sur processeurs peu puissants dépourvus de GPU dédié.

4.1.2 Mise en place d'un package ROS2

L'intégration du modèle de détection dans l'écosystème ROS2 du robot s'est faite au travers de la création d'un package dédié, detection_pkg, de type ament_python*, généré à l'aide de la commande ros2 pkg create. Ce choix de type de package est motivé par l'utilisation de bibliothèques Python (Ultralytics, OpenVINO, OpenCV*, cv_bridge), plus adaptées à ce cas d'usage qu'un package C++.

4.2 Détection 2D

4.2.1 Fonctionnement du noeud ROS2 2D

L'intégration du modèle 2D dans l'écosystème du robot s'effectue au travers du package detection_pkg via un nœud ROS2 dédié. Le schéma bloc ci-dessous illustre le cheminement complet des données depuis la capture vidéo jusqu'à la publication des messages ROS.

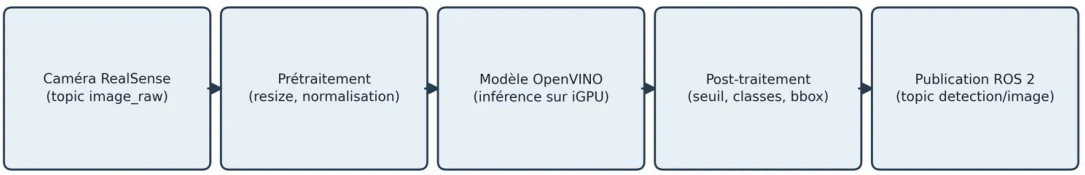


Figure 4 : Fonctionnement du noeud ROS2 2D

4.3 Détection 3D

4.3.1 Problématique du passage de la 2D à la 3D et synchronisation multi-capteurs

La détection 2D fournit la position géométrique d'un objet en coordonnées pixels (u, v) au sein du plan image. Cependant, pour permettre au robot mobile d'interagir avec son environnement (évitement d'obstacles, arrêt devant un feu de circulation ou suivi de balises), la connaissance de la position spatiale métrique (X, Y, Z) exprimée dans le repère réel tri-dimensionnel du robot est indispensable.

L'extraction d'une position 3D à partir du capteur Intel RealSense D435 impose de résoudre deux contraintes majeures :

- La synchronisation temporelle des flux : La caméra produit simultanément un flux couleur RGB et un flux de profondeur Depth. Le nœud de détection 3D doit s'assurer d'associer une image couleur reçue à l'instant t avec la carte de profondeur exactement mesurée à ce même instant t afin d'éviter tout décalage spatial lorsque le robot est en mouvement
- L'alignement géométrique des plans (Depth-to-Color Alignment) : Les capteurs infrarouges (mesurant la profondeur) et le capteur RGB sont séparés physiquement par une distance d'entraxe (la baseline B). Leurs foyers optiques ne sont pas confondus. Pour superposer un pixel (u, v) de l'image couleur avec son équivalent exact sur la carte de profondeur, le driver RealSense applique une transformation géométrique inverse basée sur les paramètres d'usine.

4.3.2 Mathématiques, Projections et Optimisations Algorithmiques

4.3.2.1 Modélisation géométrique : Le modèle du sténopé

Le modèle mathématique régissant la transformation d'un point de l'espace 3D vers le plan image 2D repose sur la géométrie de la caméra sans lentille.

Soit un point P représenté par son vecteur de coordonnées cartésiennes $P = [Xc\ Yc\ Zc]$ dans le repère optique de la caméra. Sa projection $p = [u_c\ v_c]$ en coordonnées pixels sur le plan image 2D est définie par la relation matricielle

$$s \cdot \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \cdot \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix}$$

Où :

- $s = Z_c$ représente le facteur d'échelle (qui correspond à la profondeur axiale du point)
- K est la matrice intrinsèque de la caméra (extraite directement du topic ROS /camera/camera/color/camera_info)
- f_x, f_y sont les distances focales exprimées en pixels suivant les axes X et Y
- c_x, c_y sont les coordonnées du centre optique dans le système de coordonnées de l'image (en pixels)

En développant ce système d'équations matricielles, les coordonnées pixels (u, v) sont exprimées par :

$$u = f_x \cdot \frac{X_c}{Z_c} + c_x$$

$$v = f_y \cdot \frac{Y_c}{Z_c} + c_y$$

4.3.2.2 La rétro-projection 2D → 3D

Pour déterminer la position 3D $P_c(X_c, Y_c, Z_c)$ d'un objet repéré par sa Bounding Box* 2D, il faut inverser le modèle géométrique du sténopé*

En supposant que la valeur de profondeur mesurée à l'emplacement du pixel (u, v) est connue via la carte de profondeur alignée, désignée par $D(u, v) = Z_c$ (exprimée en mètres après conversion) on a :

- Isoler X_c à partir de l'équation de u :

$$u - c_x = f_x \cdot \frac{X_c}{Z_c} \Rightarrow X_c = \frac{(u - c_x) \cdot Z_c}{f_x}$$

- Isoler Y_c à partir de l'équation de v :

$$v - c_y = f_y \cdot \frac{Y_c}{Z_c} \Rightarrow Y_c = \frac{(v - c_y) \cdot Z_c}{f_y}$$

- La profondeur est directement extraite :

$$Z_c = D(u, v)$$

4.3.2.3 Extraction robuste de la profondeur dans la ROI et Filtrage Médian

L'extraction simpliste de la profondeur au niveau du seul pixel central $p_{center} = \left(\frac{u_{min}+u_{max}}{2}, \frac{v_{min}+v_{max}}{2}\right)$ de la Bounding Box 2D présente une vulnérabilité opérationnelle majeure :

- Si le centre de l'objet comporte un trou, une surface hautement réfléchissante, ou si la mesure stéréoscopique échoue à cet endroit, la valeur renvoyée est égale à 0 ou NaN
- Si l'objet possède une géométrie fine (ex. un piquet de panneau), le pixel central peut pointer vers l'arrière-plan situé plusieurs mètres derrière

Pour garantir une estimation robuste de Z_{objet}, une méthode d'analyse statistique sur Région d'Intérêt Réduite (ROI*) avec Filtrage Médian a été développée :

- Réduction spatiale de la Bounding Box : afin d'éliminer les pixels d'arrière-plan figurant sur les bordures de la boîte englobante, la région d'extraction est resserrée au centre de l'objet via un facteur d'échelle

$$\Delta u = u_{\max} - u_{\min}, \quad \Delta v = v_{\max} - v_{\min}$$

$$u_{\text{start}} = u_{\min} + \frac{1-\alpha}{2} \cdot \Delta u, \quad u_{\text{end}} = u_{\max} - \frac{1-\alpha}{2} \cdot \Delta u$$

$$v_{\text{start}} = v_{\min} + \frac{1-\alpha}{2} \cdot \Delta v, \quad v_{\text{end}} = v_{\max} - \frac{1-\alpha}{2} \cdot \Delta v$$

- Filtrage des valeurs aberrantes et calcul de la médiane : la valeur finale de la profondeur attribuée à l'objet, Z_{objet}, correspond à la médiane de cet ensemble de valeurs

Justification de l'estimateur médian : contrairement à la moyenne arithmétique qui subit une déviation importante en présence de valeurs aberrantes, la médiane assure une excellente stabilité. Même si une partie des valeurs est fausse ou aberrante, la profondeur mesurée reste calée sur la surface de l'objet.

4.3.2.4 Calcul des dimensions réelles 3D

En utilisant la valeur de profondeur robuste Z_{objet} calculée précédemment, il est possible d'estimer les dimensions physiques réelles (exprimées en mètres) de l'objet détecté à partir de sa valeur 2D en pixels

$$W_{3D} = \frac{(u_{\max} - u_{\min}) \cdot Z_{\text{objet}}}{f_x}$$

$$H_{3D} = \frac{(v_{\max} - v_{\min}) \cdot Z_{\text{objet}}}{f_y}$$

4.3.3 Architecture des topics ROS 2 publiés

Le nœud de détection 3D publie ses résultats sur trois topics principaux, assurant les échanges avec les nœuds de décision et de contrôle (machine à états du robot) :

- le topic /detection/image_3d de type sensor_msgs/msg/Image : il contient les images couleur annotées avec les boîtes englobantes 2D, les étiquettes de classes et les distances 3D calculées
- le topic /detection/position_3d de type geometry_msgs/msg/PointStamped qui contient la position 3D (X, Y, Z) de chaque objet détecté dans le repère caméra
- enfin de topic /detection/metadata de type vision_msgs/msg/Detection2DArray* qui contient l'ensemble structuré des détections : classe, score de confiance, position et dimensions 3D de chaque objet, destiné à être consommé par le nœud de machine à états développé par Matthias.

Intelligence artificielle

5.1 Data Engineering et Datasets

5.1.1 Dataset BDK100

Pour permettre au robot de détecter l'environnement routier, un premier jeu de données volumineux issu de la recherche a été exploité : le dataset BDD100k (Berkeley DeepDrive 100K). Ce dataset comprend 100 000 images de scènes de conduite réelles associées à des annotations JSON*.

Lors des premiers essais de conversion, un bug de résolution de chemin d'accès entraînait l'oublie quasi totale du dataset : le script s'arrêtait à 1156 images converties sur 70000, ignorant le reste en raison de l'arborescence du dataset.

Ce problème a été résolu en développant un algorithme d'indexation mémoire préalable utilisant la recherche récursive. Le script mappe directement chaque nom d'image avec son chemin absolu sur le disque avant de procéder au parsing des annotations JSON.

Pour chaque Bounding Box 2D définie dans le JSON par ses coins extrêmes $[x_1, y_1, x_2, y_2]$ et pour une image de dimensions $W \times H$ (où $W = 1280$ et $H = 720$ pixels), la conversion vers le format YOLO applique la normalisation suivante :

$$c_x = \frac{x_1 + x_2}{2 \cdot W}, \quad c_y = \frac{y_1 + y_2}{2 \cdot H}$$

$$w = \frac{x_2 - x_1}{W}, \quad h = \frac{y_2 - y_1}{H}$$

Où (c_x, c_y) représentent le centre de la boîte et (w, h) ses dimensions, toutes exprimées sous forme de ratios compris entre $0,0$ et $1,0$. Les annotations sont ensuite enregistrées sous forme de fichiers texte .txt individuels et associées au fichier de configuration bdd100k.yaml.

5.1.2 Création d'un dataset personnalisé (Roboflow)

5.1.2.1 Utilisation de Roboflow et nécessité d'un dataset sur-mesure

Bien que le modèle entraîné sur BDD100k fournisse de bons résultats sur des routes réelles, son application sur le robot mobile au sein du circuit de test a révélé un problème de décalage de domaine. Le circuit conçu dans la salle de projet utilise des

panneaux de signalisation miniatures (Cédez le passage, Stop, Passage piéton, Obligation de tourner à droite) ainsi que des feux tricolores miniatures.

Les caractéristiques visuelles de ces éléments miniatures (texture du plastique, reflets sous l'éclairage néon intérieur, proportions particulières) différaient trop du dataset BDD100k. Une phase d'acquisition et d'annotation dédiée a donc été menée via l'outil Roboflow.

La constitution du dataset personnalisé a débuté par une phase d'acquisition photographique : plus de 150 clichés des éléments miniatures du circuit (panneaux de signalisation et feux tricolores) ont été pris sous différents angles et conditions d'éclairage, afin de capturer la variabilité visuelle que le modèle devra reconnaître en conditions réelles.

Ces images ont ensuite été importées sur la plateforme Roboflow, un outil en ligne de labellisation et de gestion de datasets pour la vision par ordinateur. Chaque image a été annotée manuellement en traçant une boîte englobante (bounding box) autour de chaque objet. Roboflow propose également un outil d'assistance par intelligence artificielle capable de proposer automatiquement des boîtes englobantes à partir d'une description textuelle en anglais de l'objet (par exemple Crosswalk sign, Yield sign, Traffic light, Stop sign).

5.1.2.2 Évolution de la classification

- Séparation des classes de feux : Initialement, une classe unique feu regroupait toutes les photos. Le modèle présentait de fortes hésitations et prédisait des feux de manière instable sur le décor. Le dataset a donc été ré-annoté en séparant le feu tricolore en trois classes distinctes : feu_rouge, feu_orange et feu_vert.
- Gestion des versions dans Roboflow : Lors des itérations de correction du jeu de données, un comportement de l'interface Roboflow a été identifié : toute suppression ou modification d'images n'est répercutée dans l'archive d'export que si l'utilisateur génère explicitement une nouvelle Version du dataset. Sans cette action, l'export réutilise l'ancien cache comprenant des images obsolètes.
-

5.2 Entraînement de YOLOv11

L'entraînement du modèle YOLOv11n a été réalisé localement, sur la carte graphique NVIDIA RTX portable de mon ordinateur, via la bibliothèque Ultralytics en Python. Un script d'entraînement dédié configure l'ensemble des paramètres : nombre d'époques, taille des images, taille de batch, etc, ainsi que les techniques de régularisation et d'augmentation de données présentées dans la sous partie 4.2.1. Ce script a fait l'objet de nombreuses itérations pour corriger des dysfonctionnements successifs : une première tentative d'entraînement sur un modèle YOLOv11n-seg s'est avérée incompatible avec un dataset dépourvu de masques de segmentation. Une reprise d'entraînement interrompu a nécessité l'ajout du paramètre resume=True pointant vers les derniers poids sauvegardés. Enfin, un phénomène de sur-apprentissage* (le modèle mémorisait le décor de la salle plutôt que les objets eux-mêmes) a été corrigé en renforçant les paramètres d'augmentation de données et en réduisant le nombre d'époques. Chaque entraînement sauvegarde automatiquement une copie des poids (.pt) à intervalle régulier (save_period), ce qui a permis de

sélectionner a posteriori l'époque offrant le meilleur compromis entre performance et généralisation. Une fois l'entraînement terminé, le modèle a été transféré vers le robot par SCP* (ou gitclone), puis converti au format OpenVINO directement dans le conteneur Docker, avant d'être placé dans le dossier models du package detection_pkg en remplacement du modèle précédent.

5.2.1 Techniques de régularisation et d'augmentation de données

- mosaic*=1.0 : Combine quatre images d'entraînement en une seule, forçant le modèle à détecter les objets dans des contextes spatiaux variés
- mixup*=0.2 : Superpose deux images en transparence, détruisant la cohérence de l'arrière-plan
- scale=0.6 : Applique un zoom/dézoom aléatoire détachant les panneaux miniatures de la paroi murale située derrière
- Pondération de classification : La fonction de perte (loss function) de classification a été multipliée par 6 par rapport à sa valeur par défaut ($0,5$). Cela force le réseau à pénaliser fortement les erreurs de confusion de classes (notamment sur les feux de circulation)

5.2.2 Optimisation et Déploiement (OpenVINO)

Pour exécuter le réseau de neurones sur le processeur embarqué du robot sans GPU dédié, le modèle PyTorch entraîné (.pt) a été converti vers le format Intel OpenVINO

Modalités de conversion et gain de performance :

- Format FP16* (half=True) : La conversion réduit la précision des poids des flottants 32 bits (FP32) vers des flottants 16 bits. Cette opération divise par deux la taille du modèle sur le disque sans dégradation mesurable de la précision de détection (mAP *), tout en doublant la vitesse de transfert mémoire.
- Structure générée : L'export génère un dossier d'un modèle OpenVINO contenant deux fichiers fondamentaux :
- Le fichier .xml : Décrit la topologie géométrique du graphe de calcul du réseau de neurones
- Le fichier .bin : Stocke la matrice binaire des poids optimisés
- Résultat opérationnel : Grâce au pont logiciel mis en place dans le conteneur Docker (pilotes Level Zero et OpenCL configurés au Chapitre 2), le nœud ROS2 est exécuté avec l'argument device:=GPU. L'inférence est ainsi déportée sur le processeur graphique intégré Intel (iGPU), permettant d'obtenir une cadence d'analyse temps réel fluide.

5.2.3 analyse de la couleur des feux tricolores

La détection des feux tricolores a soulevé une difficulté spécifique : un premier entraînement regroupant toutes les couleurs de feu sous une classe unique a conduit le modèle à confondre systématiquement les états rouge, orange et vert. De plus, le

modèle reconnaissait de manière aléatoire le décor comme étant un feu rouge. La séparation du dataset en trois classes distinctes (feu_rouge, feu_orange, feu_vert) a résolu la confusion de détection, mais au prix d'une difficulté à discriminer correctement la couleur, en particulier dans des conditions de luminosité variables.

La solution retenue à été de filtrer la couleur des feux directement dans le code python via l'utilisation de OpenCV. Les trois classes de feux ont été regroupées en une seule, la différenciation des couleurs se fait maintenant grâce à une analyse de la densité et de la teinte des pixels qui composent le feu. Cette approche en deux temps permet une détection et une classification robuste.

Tests, Validation et Résultats

6.1 Construction de la piste de test

Afin de valider le pipeline* de détection en conditions réelles, une piste de test a été construite en intérieur, dans la salle de projet du laboratoire, en collaboration avec Thomas Chavaroche. Le tracé de la route a été matérialisé au sol à l'aide de ruban adhésif jaune, complété par des panneaux de signalisation miniatures (Cédez le passage, Stop, passage piéton, obligation de tourner à droite) ainsi que des feux tricolores miniatures positionnés aux points stratégiques du circuit. Cette piste a constitué le terrain d'expérimentation principal pour l'ensemble des développements présentés dans ce rapport, aussi bien pour la constitution du dataset Roboflow (cf. partie 5) que pour la validation finale du test de détection.

6.2 Validation de la détection en conditions réelles

Le fonctionnement de l'algorithme de détection a été validé en conditions réelles sur la piste, en visualisant le flux annoté publié sur le topic /detection/image_3d via l'interface RViz2, accessible à distance par le serveur NoVNC configuré en section 2.1.1.

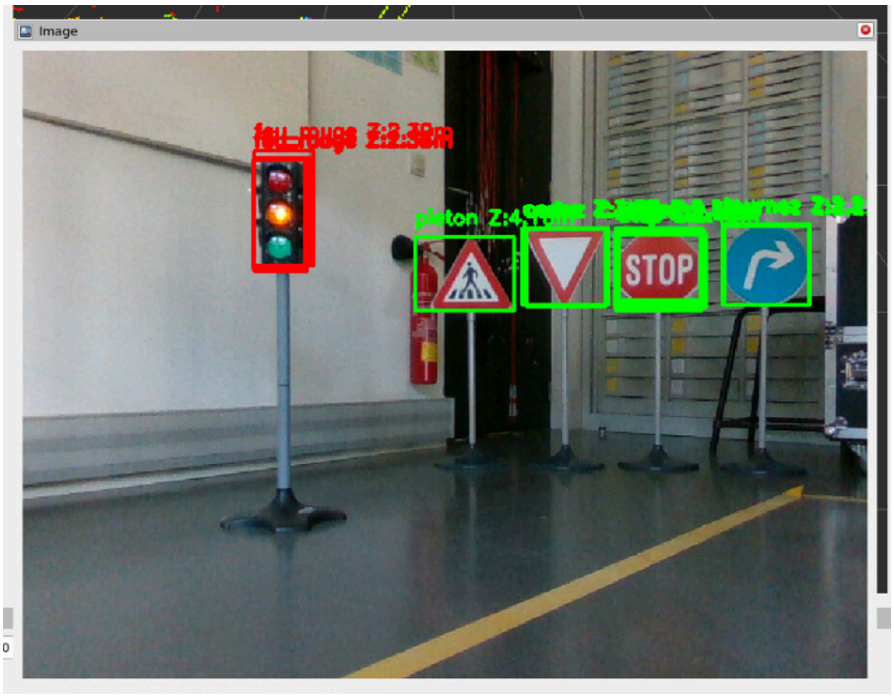


Figure 5 : Résultats de la détection 3D sur NoVNC

6.3 Résultats du dataset Roboflow

Les figures suivantes illustrent le processus de labellisation et les résultats obtenus sur la plateforme Roboflow, et sur l'entraînement du modèle.

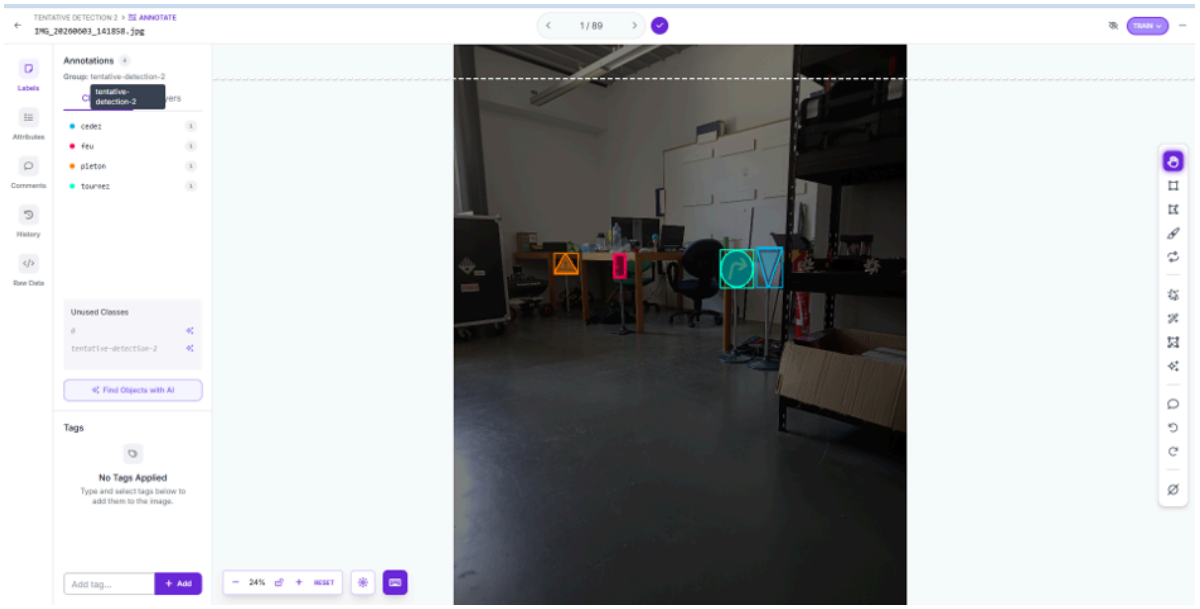


Figure 6 : Labellisation sur Roboflow

```
40 epochs completed in 0.061 hours.
Optimizer stripped from C:\Users\maxim\Desktop\bdd100k_training\runs\roboflow3_anti_bias3\weights\last.pt, 5.5MB
Optimizer stripped from C:\Users\maxim\Desktop\bdd100k_training\runs\roboflow3_anti_bias3\weights\best.pt, 5.5MB

Validating C:\Users\maxim\Desktop\bdd100k_training\runs\roboflow3_anti_bias3\weights\best.pt...
Ultralytics 8.3.248  Python-3.11.9 torch-2.10.0+cu128 CUDA:0 (NVIDIA GeForce RTX 3070 Laptop GPU, 8192MiB)
YOLO11n summary (fused): 100 layers, 2,583,517 parameters, 0 gradients, 6.3 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 8/8 5.2it/s 1.5s
                   all        114        118      0.832      0.859      0.892      0.758
                 cedez          9          9          1      0.824      0.912      0.798
            feu_orange         13         13      0.744      0.846      0.883       0.64
             feu_rouge         16         16      0.627       0.63      0.622      0.454
              feu_vert         14         14      0.681      0.714      0.848      0.593
                pieton         18         18      0.877          1      0.995       0.93
                  stop         25         25       0.98          1      0.995      0.933
               tournez         23         23      0.917          1      0.988       0.96
Speed: 0.2ms preprocess, 4.9ms inference, 0.0ms loss, 1.3ms postprocess per image
Results saved to C:\Users\maxim\Desktop\bdd100k_training\runs\roboflow3_anti_bias3
Entraînement terminé ! Le modèle corrigé est prêt.
```

Figure 7 : Résultats de l'entraînement

6.4 Tests finaux et présentation

À l'issue du stage, une série de tests finaux a été menée par les trois membres de l'équipe, Thomas Chavaroche, Matthias Vermot-Desroches et moi-même, sur la piste complète, afin de valider l'intégration de l'ensemble des briques développées : détection 2D/3D, remontée des métadonnées vers la machine à états, et comportement du robot face aux différents éléments de signalisation ainsi que suivi des lignes. Ces travaux ont été présentés lors d'une soutenance devant le personnel de Polytech Nice-Sophia, permettant de restituer l'ensemble de la démarche technique et pédagogique menée au cours du stage.

Ingénierie Pédagogique et Transmission

7.1 L'objectif pédagogique

L'intégration d'un pipeline complet alliant intelligence artificielle, vision par ordinateur 3D et robotique mobile sous ROS2 constitue un défi technique majeur. L'objectif principal de la démarche pédagogique a été de déconstruire cette complexité en une suite d'exercices pratiques et progressifs, accessibles à des élèves ingénieurs sans pour autant sacrifier la rigueur théorique.

Le but principal est de permettre aux étudiants de manipuler la chaîne complète, depuis la prise de vue sur la piste jusqu'au contrôle du robot en réaction aux détections.

J’ai conçu quatre travaux pratiques au cours du stage :

- Mise en place de l'environnement : configuration de l'accélération matérielle OpenVINO sur l'iGPU Intel, résolution des problèmes de permissions Docker (GID du groupe render), mise en cache persistante des dépendances.
- Détection d'objets 2D : implémentation d'un nœud ROS2 de détection YOLOv11 exploitant le flux couleur de la caméra RealSense.

- Détection d'objets 3D : extension du TP précédent par l'exploitation de la carte de profondeur, avec rétro-projection 2D→3D et estimation des dimensions réelles des objets détectés.
- Création d'un dataset personnalisé avec Roboflow : prise en main de l'outil de labellisation, constitution d'un jeu de données sur-mesure, export et fine-tuning d'un modèle YOLOv11 pré-entraîné.

7.2 Création des supports

La rédaction de l'intégralité des supports théoriques et pratiques a été réalisée sous Overleaf* (LaTeX*), garantissant un formalisme mathématique et une structure documentaire conformes aux standards académiques.

7.3 Développement de l'environnement étudiant

Afin d'équilibrer l'effort de programmation et de garantir que les travaux pratiques soient réalisables dans le temps imparti, un environnement de développement adapté a été mis en place.

7.3.1 Squelettes de code à trous

Les codes fournis aux étudiants contiennent l'ensemble de la structure ROS2 fonctionnelle. Seules les fonctions clés liées au traitement de vision, aux calculs géométriques et aux abonnements aux topics, publication des messages sont laissées vides.

7.3.2 Chaîne d'outils et scripts d'automatisation

L'ensemble des étapes de configuration et de déploiement (installation des dépendances, modification du Dockerfile et du script build_and_run.sh, export et transfert des modèles) a été rédigé sous forme de tutoriels pas-à-pas, disponibles avec l'intégralité du code source, des fichiers de configuration et des sujets de TP sur le dépôt GitHub* du projet.

7.4 Bêta-testing et validation

Avant la mise en service officielle auprès des étudiants, les travaux pratiques ont été soumis à une phase de bêta-testing en conditions réelles par d'autres étudiants du laboratoire (comme par exemple Matthias et Thomas).

Retours d'expérience et exemples d'ajustements apportés :

- Gestion des dépendances ROS2 : Les essais ont mis en évidence l'absence de certains paquets système au sein de l'environnement Docker de base (notamment ros-humble-vision-msgs). Les scripts de configuration ainsi que le fichier package.xml ont été mis à jour pour automatiser leur installation

- Robustesse de l'interfaçage : Les tests d'intégration avec le nœud de machine à états développé par Matthias ont permis de valider la structure des données publiées. Le topic /detection/metadata fournissant des structures de type vision_msgs/msg/Detection2DArray a ainsi été définitivement adopté pour transmettre de manière fiable la classe, la confiance et la position 3D de chaque élément détecté.
- Clarification des supports : Les remarques issues des bêta-tests ont servi à affiner le texte des sujets sur Overleaf, en clarifiant les instructions de configuration réseau (SSH, NoVNC) et les étapes de transfert des poids OpenVINO

Conclusion

Ce stage d'assistant ingénieur a permis de faire passer le robot AgileX Scout Mini d'un matériel brut, tout juste acquis par le laboratoire, à une plateforme documentée et fonctionnelle, capable de détecter en temps réel des objets d'intérêt routier en trois dimensions grâce à l'accélération matérielle Intel OpenVINO.

Au-delà de l'aspect purement technique, ce travail a nécessité de résoudre de nombreux problèmes d'intégration système (permissions Docker, pilotes GPU, réseau) rarement documentés, ainsi que de constituer un jeu de données sur mesure adapté à un environnement de test miniature, sensiblement différent des données de conduite réelle disponibles publiquement.

La dimension pédagogique du projet a occupé une place centrale : quatre travaux pratiques ont été conçus, testés par mes collègues de stage, puis corrigés à partir de leurs retours, avec pour objectif de permettre aux futurs étudiants du département de s'approprier une chaîne complète de robotique mobile, de la prise en main matérielle jusqu'à l'intelligence artificielle embarquée, de manière autonome et progressive.

Les tests finaux menés conjointement avec Thomas Chavaroche et Matthias Vermot-Desroches, ainsi que la présentation devant le personnel de Polytech Nice-Sophia, ont permis de valider concrètement l'ensemble de cette démarche.

Ce stage a été l'occasion de développer des compétences aussi bien en robotique et en vision par ordinateur qu'en administration système bas niveau, tout en découvrant les enjeux propres à la transmission pédagogique d'un sujet technique complexe.

Annexe

Lien du dépot GitHub :
https://github.com/MatthiasVermot-Desroches/Polytech-Scout-Agilex-GR-TP

Glossaire

ament_python — Type de package ROS 2 destiné aux nœuds écrits en Python, par opposition à ament_cmake pour le C++.

apt (Advanced Package Tool) — Gestionnaire de paquets logiciels des systèmes Ubuntu/Debian, utilisé pour installer, mettre à jour ou supprimer des programmes système.

Baseline (ligne de base) — Distance physique séparant deux capteurs d'un système stéréoscopique, utilisée dans le calcul de la profondeur par disparité.

Bounding box (boîte englobante) — Rectangle délimitant la zone d'un objet détecté dans une image, défini par les coordonnées de ses coins.

BSSID — Adresse identifiant un point d'accès Wi-Fi, utilisée pour se connecter au routeur Teltonika du robot.

Conteneur (Docker) — Environnement d'exécution isolé et portable, embarquant ses propres dépendances logicielles indépendamment du système hôte sur lequel il tourne.

CPU (Central Processing Unit / processeur) — Composant électronique effectuant les calculs généraux d'un ordinateur ; par opposition au GPU, il traite les instructions de façon séquentielle et n'est pas optimisé pour les calculs massivement parallèles.

cv_bridge — Bibliothèque ROS2 assurant la conversion entre les images au format ROS et le format utilisé par OpenCV.

Detection2DArray — Type de message ROS2 (issu du paquet vision_msgs) structurant un ensemble de détections d'objets (classe, score, position).

Disparité — Décalage horizontal entre la position d'un même point observé par deux capteurs stéréoscopiques.

DNS (Domain Name System) — Système traduisant les noms de domaine (ex. github.com) en adresses IP ; des instabilités DNS peuvent bloquer les téléchargements lors de la construction d'un conteneur Docker.

Docker — Logiciel permettant de créer, exécuter et distribuer des conteneurs.

Dockerfile — Fichier texte décrivant les instructions nécessaires à la construction d'une image Docker (système, dépendances, configuration).

FabLab — Atelier collaboratif équipé d'outils de fabrication numérique (imprimantes 3D, découpe laser, etc.), ouvert aux étudiants et chercheurs.

FP16 / FP32 — Formats de représentation numérique en virgule flottante sur 16 bits (demi-précision) ou 32 bits (simple précision) ; le FP16 réduit la taille d'un modèle au prix d'une précision moindre.

FPS (Frames Per Second / images par seconde) — Nombre d'images traitées ou affichées chaque seconde, indicateur de fluidité d'un flux vidéo ou d'un pipeline de détection.

Fusion 360 — Logiciel de conception assistée par ordinateur (CAO), utilisé ici pour modéliser le support de caméra imprimé en 3D.

GID (Group ID) — Identifiant numérique d'un groupe d'utilisateurs sous Linux, utilisé par le système pour les contrôles d'accès aux périphériques.

GitHub — Plateforme en ligne d'hébergement de code source, utilisée ici pour stocker l'ensemble du code, de la configuration et des sujets de TP du projet.

GNSS (Global Navigation Satellite System) — Terme générique désignant les systèmes de positionnement par satellites.

GPU (Graphics Processing Unit / processeur graphique) — Composant électronique optimisé pour effectuer un très grand nombre de calculs simples en parallèle, particulièrement adapté à l'inférence de réseaux de neurones.

iGPU (Integrated GPU / GPU intégré) — Processeur graphique intégré directement au CPU, par opposition à une carte graphique dédiée (GPU discret).

Image Docker — Modèle figé et réutilisable à partir duquel un conteneur est créé, contenant un système d'exploitation minimal et l'ensemble des logiciels préinstallés (ex. « image Ubuntu 22.04 »).

IMU (Inertial Measurement Unit / centrale inertielle) — Capteur mesurant l'accélération et la vitesse angulaire, utilisé pour estimer l'orientation d'un système.

JSON (JavaScript Object Notation) — Format de fichier texte structuré, très utilisé pour stocker ou échanger des données.

LaTeX — Système de préparation de documents utilisé pour la rédaction du rapport et des supports de TP, particulièrement adapté à la mise en forme de contenus mathématiques.

Level Zero — Interface bas niveau d'Intel permettant un accès direct aux accélérateurs matériels (GPU, NPU) pour le calcul haute performance.

LIDAR (Light Detection And Ranging) — Capteur de télémétrie par laser, permettant de mesurer les distances et de cartographier l'environnement en trois dimensions.

Middleware — Logiciel intermédiaire assurant la communication entre différents composants d'un système sans que chacun ait besoin de connaître les détails des autres, ROS2 est le middleware utilisé pour faire communiquer les différents programmes du robot (caméra, détection, navigation, etc.).

mAP (mean Average Precision) — Métrique standard d'évaluation des modèles de détection d'objets, mesurant la précision moyenne sur l'ensemble des classes.

Matrice intrinsèque — Matrice de calibration propre à une caméra (distances focales, centre optique), utilisée pour convertir des coordonnées 3D en coordonnées pixels.

Mosaic / Mixup — Techniques d'augmentation de données en apprentissage profond : la première combine plusieurs images en une seule, la seconde les superpose en transparence, afin de diversifier artificiellement le jeu d'entraînement.

navsat_transform — Nœud du framework robot_localization convertissant des coordonnées GPS en coordonnées cartésiennes locales.

Nœud (ROS2) — Programme exécutable élémentaire dans un système ROS 2, communiquant avec les autres nœuds via des topics.

NoVNC — Protocole et outil permettant d'accéder à distance, via un navigateur web, au bureau graphique d'une machine distante.

NPU (Neural Processing Unit) — Unité de calcul dédiée à l'accélération des réseaux de neurones, présente sur certains processeurs récents.

NTRIP (Networked Transport of RTCM via Internet Protocol) — Protocole de diffusion des corrections RTK par Internet.

Odométrie — Estimation du déplacement d'un robot à partir de la mesure de la rotation de ses roues.

OpenCL — Interface de programmation standard permettant d'exploiter des architectures de calcul (CPU, GPU, autres accélérateurs).

OpenCV — Bibliothèque logicielle open-source de traitement d'image et de vision par ordinateur.

OpenVINO — Kit de développement (SDK) d'Intel destiné à optimiser et déployer des modèles d'intelligence artificielle sur du matériel Intel (CPU, iGPU, NPU).

Overfitting (sur-apprentissage) — Phénomène par lequel un modèle mémorise les particularités de son jeu d'entraînement plutôt que d'apprendre à généraliser, dégradant ses performances sur des données nouvelles.

Overleaf — Éditeur LaTeX en ligne collaboratif, utilisé pour la rédaction du rapport et des supports de TP.

Package (ROS2) — Unité de base d'organisation du code dans ROS 2, regroupant nœuds, fichiers de configuration et de lancement autour d'une fonctionnalité donnée.

PETG — Matériau plastique utilisé en impression 3D, offrant un bon compromis entre résistance mécanique et facilité d'impression.

Pipeline — Enchaînement ordonné d'étapes de traitement, chacune prenant en entrée le résultat de la précédente (ex. le pipeline de détection : capture → prétraitement → inférence → post-traitement → publication).

PPA (Personal Package Archive) — Dépôt de paquets logiciels tiers pour les distributions Ubuntu/Debian, permettant d'installer des versions non incluses dans les dépôts officiels.

Python — Langage de programmation utilisé pour l'ensemble des scripts d'entraînement, de traitement d'image et des nœuds ROS 2 de détection.

RGB / RGB-D — RGB désigne une image couleur classique (rouge, vert, bleu) ; RGB-D désigne une caméra fournissant en plus une carte de profondeur (Depth) associée à chaque pixel.

ROI (Region of Interest / Région d'Intérêt) — Sous-partie d'une image sur laquelle un traitement spécifique est appliqué.

ROS2 (Robot Operating System) — Ensemble d'outils et de bibliothèques logicielles (middleware) permettant de structurer la communication entre les différents programmes d'un système robotique.

RTCM3 (Radio Technical Commission for Maritime services, version 3) — Format standardisé des messages de correction différentielle utilisés en RTK.

RTK (Real-Time Kinematic) — Technique de positionnement satellite utilisant des corrections différentielles pour atteindre une précision centimétrique.

SCP (Secure Copy Protocol) — Protocole de transfert de fichiers sécurisé entre machines.

SDK (Software Development Kit / kit de développement logiciel) — Ensemble d'outils, de bibliothèques et de documentation fourni pour développer des programmes exploitant une technologie donnée.

SSH (Secure Shell) — Protocole permettant une connexion et un contrôle à distance sécurisés d'une machine.

Sténopé — Modèle géométrique simplifié de formation d'image (camera obscura sans lentille), à la base des équations de projection utilisées en vision par ordinateur.

Teltonika — Marque du routeur industriel Wi-Fi/4G utilisé pour la connexion réseau du robot.

Topic (ROS2) — Canal de communication nommé permettant l'échange asynchrone de messages entre les nœuds d'un système ROS 2.

Ubuntu — Distribution du système d'exploitation Linux, utilisée à la fois sur la machine hôte du robot et dans le conteneur Docker.

Ultralytics — Bibliothèque logicielle Python open-source implémentant les modèles de la famille YOLO.

vision_msgs / sensor_msgs / geometry_msgs — Paquets ROS 2 définissant des formats de messages standardisés, respectivement pour les données de vision (détections), les données de capteurs (images) et les données géométriques (positions).

Workspace (ROS2) — Répertoire de travail regroupant l'ensemble des packages ROS2 d'un projet, compilés et utilisés ensemble.

YOLO (You Only Look Once) — Famille d'architectures de réseaux de neurones pour la détection d'objets en temps réel, opérant en une seule passe sur l'image entière.

Résumé/Abstract

Synthèse (français)

Ce rapport présente mon stage d'assistant ingénieur de quatrième année, réalisé au sein de Polytech'Lab. La mission confiée à l'origine portait sur la rédaction d'une documentation technique et la conception de travaux pratiques pour un robot mobile AgileX Scout Mini nouvellement acquis par le laboratoire. Pour construire des supports concrets et pertinents, ce travail a progressivement nécessité la mise en place d'une véritable chaîne technique : environnement Docker/ROS2, accélération matérielle de l'intelligence artificielle sur processeur Intel via OpenVINO, détection d'objets en 2D puis en 3D à partir d'une caméra de profondeur, exploration d'une localisation par GPS RTK, et constitution d'un jeu de données personnalisé pour entraîner un modèle YOLOv11 adapté à un circuit routier miniature. Cette double dynamique, développer pour documenter, documenter pour transmettre, a abouti à un pipeline de détection fonctionnel, validé en conditions réelles sur une piste construite pour l'occasion, ainsi qu'à quatre travaux pratiques testés et présentés devant le personnel de Polytech Nice-Sophia.

Summary (English)

This report presents my fourth-year engineering assistant internship, carried out at Polytech'Lab. The original mission was to write technical documentation and design hands-on labs for newly acquired AgileX Scout Mini mobile robots. Building concrete, relevant material progressively required setting up a full technical pipeline : a Docker/ROS2 environment, hardware-accelerated AI inference on an Intel processor via OpenVINO, 2D and then 3D object detection from a depth camera, an exploration of RTK GPS localization, and a custom dataset built to train a YOLOv11 model adapted to a miniature road circuit. This back-and-forth between building and documenting resulted in a functional detection pipeline, validated under real conditions on a test track built for this purpose, along with four hands-on labs that were tested and presented to Polytech Nice-Sophia staff.